ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Revista Ciencias, Sistemas y Tecnologia

SECCIÓN 3

Seguridad, despliegue y operación

¿Quién protege el sistema y responde cuando llega a producción?

ARTÍCULOS DESTACADOS

- El especialista en seguridad antes y después de la IA
- Seguridad desde el diseño y responsabilidad compartida
- Despliegues automatizados con supervisión humana
- Código y configuraciones generadas por IA
- Incidentes, observabilidad y recuperación



ENTREVISTA • PROFESIONAL

Ing. Marlon Francisco Orellana López

Coordinador del Área de Software - Docente

Automatización de procesos - Analítica de datos - Gestión tecnológica



TEMA

Automatizar no significa dejar de decidir

Agentes - riesgo - responsabilidad

Ver la entrevista

PRESENTACIÓN

Marlon Francisco Orellana López es ingeniero en sistemas, colegiado desde 2006, con alrededor de veinte años de docencia en la Facultad de Ingeniería. Coordina el Área de Software, participa en la Escuela de Posgrados y ha desarrollado su carrera privada en analítica de datos, administración de proyectos tecnológicos y automatización de procesos.

ANTES DE LA IA GENERATIVA

Antes de esta revolución de inteligencia artificial y agentes, ¿cómo era la automatización y qué procesos repetitivos dependían completamente de una persona?

Al mirar hacia los años noventa, Orellana recuerda una transición progresiva. Muchas tareas que antes vivían en papel comenzaron a digitalizarse: reportes tabulados, registros, cálculos y gráficos pasaron a apoyarse en computadoras y hojas de cálculo. Eso redujo imprecisiones y permitió manejar más información, pero la intervención humana seguía siendo fuerte. Durante años la automatización fue, en realidad, semiautomatización: la máquina ayudaba, pero una persona continuaba ejecutando, verificando o trasladando buena parte del trabajo.

El salto llegó con la estandarización de datos y procesos, el uso de la nube, la seguridad y el crecimiento del volumen de información. Con millones de registros y tiempos de respuesta más exigentes, automatizar dejó de ser comodidad y pasó a ser una necesidad para reducir costos, errores y tiempo. Allí aparecen procesos donde la intervención humana puede ser mínima, siempre que el riesgo sea bajo y el resultado pueda supervisarse.

TRANSICIÓN

Usted ubica alrededor de 2015 un cambio importante. ¿Qué impulsó ese salto hacia una automatización mucho más agresiva?

Para Orellana, el cambio no fue únicamente tecnológico. También fue económico y cultural: las organizaciones comenzaron a necesitar decisiones más rápidas y las personas empezaron a valorar de otra manera su tiempo. Si una actividad puede repetirse automáticamente, dedicar horas humanas a ejecutarla una y otra vez deja de ser una justificación suficiente del trabajo.

El objetivo pasa entonces de operar a decidir. Una persona puede dejar que el sistema procese lo repetitivo y utilizar ese tiempo para revisar resultados, detectar excepciones y tomar decisiones estratégicas. Esa presión coincide con un cambio generacional y con una economía cada vez más veloz. Automatizar, en ese contexto, significa liberar capacidad humana para tareas de mayor valor.

ENTREVISTA · CONTINUACIÓN

Ing. Marlon Francisco Orellana López

Continuación

CRITERIO

Cuando identifica un proceso repetitivo, ¿cómo decide si realmente vale la pena automatizarlo o si debería mantenerse con intervención humana?

La primera distinción debe hacerse entre ejecución y decisión. Una actividad operativa, estandarizada, repetitiva y susceptible a errores humanos es una buena candidata para automatización, especialmente si consume tiempo, incrementa costos o retrasa otras decisiones. Pero automatizar la ejecución no significa automatizar el criterio.

Orellana insiste en que el filtro humano no debería desaparecer. Un sistema puede procesar información con enorme rapidez y entregar un resultado, pero alguien debe revisar si ese resultado tiene sentido dentro del contexto de la organización. Mientras más difícil sea revertir una acción o mayor sea su impacto, más importante se vuelve conservar una instancia humana de revisión.

RESPONSABILIDAD

¿Qué decisiones sí pueden quedar completamente en manos de una automatización y cuáles deberían seguir siendo humanas?

Su respuesta establece una frontera basada en riesgo. Las decisiones sin intervención humana deberían limitarse a escenarios de bajo impacto, donde un error sea pequeño, controlable, reversible y no comprometa vidas, patrimonio, reputación ni variables críticas. Allí la autonomía puede ser razonable porque la falla es aceptable y manejable.

La situación cambia cuando hay consecuencias económicas importantes, efectos sobre personas o reputación institucional. En esos casos la decisión debe permanecer en manos humanas. Orellana pone como ejemplo una llamada de emergencia: una máquina puede clasificar palabras y señales, pero comprender plenamente violencia, accidente o peligro exige una riqueza contextual que todavía vuelve indispensable al ser humano.

CRITERIO

¿Qué cambia cuando una automatización deja de seguir reglas predefinidas y comienza a interpretar información o decidir qué acción ejecutar?

Allí aparece uno de los riesgos centrales de los agentes actuales. En una automatización tradicional se busca un comportamiento determinístico: dadas ciertas condiciones, el sistema debe llegar a un final conocido y aceptable. Cuando un agente explora caminos, interpreta información y elige acciones por sí mismo, deja de ser evidente qué peso asignará a cada variable y qué considerará un acierto o un desacierto.

La respuesta de Orellana es establecer límites. Un agente necesita fronteras claras, tolerancias y condiciones que acoten su comportamiento. Si permanece dentro de esos límites, la autonomía puede ser útil; si se le deja el camino completamente abierto, el equipo pierde capacidad de predecir las consecuencias.

RESPONSABILIDAD

Si un agente o una IA comete un error, ¿el sesgo pertenece a la creación o al creador que definió sus límites?

Orellana sitúa la responsabilidad en quien diseña y permite el margen de acción. Si el creador estableció límites, tolerancias y condiciones de error, también definió hasta dónde podía desviarse el sistema. Cuando un agente actúa dentro de un rango autorizado, la responsabilidad no puede trasladarse simplemente a la herramienta: ese comportamiento fue posible porque alguien lo permitió.

El riesgo aumenta cuando se entrega autonomía sin fronteras suficientes. Un sistema abierto puede producir resultados impredecibles o utilizarse incluso con fines destructivos. Los límites, por tanto, no son una restricción accidental: son parte esencial de la ingeniería. Automatizar puede significar delegar ejecución; no significa delegar responsabilidad.

CIERRE

Automatizar tiene sentido cuando desplaza esfuerzo operativo hacia trabajo estratégico. El reto ya no es demostrar que una máquina puede ejecutar una tarea, sino definir qué puede hacer, hasta dónde puede llegar y cuándo debe devolver la decisión a una persona.

ARTÍCULO · ESTUDIANTE

La seguridad no se agrega al final: una responsabilidad compartida desde el diseño

Autores: Daniel Eduardo Izas Marroquin; Alan Rodrigo Pamal De León

Palabras clave: Seguridad desde el diseño, responsabilidad compartida, desarrollo seguro, gestión de riesgos, ciclo de vida del software

Introducción

La seguridad del software ya no puede considerarse una actividad que se realiza únicamente

después de programar un sistema. Las vulnerabilidades pueden originarse desde la definición

de los requerimientos, la selección de la arquitectura o la configuración inicial del producto.

Por ello, la seguridad desde el diseño propone identificar riesgos y establecer controles desde

las primeras decisiones del proyecto, en lugar de corregir los problemas en producción.

Este enfoque requiere reconocer que la protección del sistema no depende únicamente del

especialista en seguridad. Analistas, arquitectos, programadores, encargados de pruebas y

operadores toman decisiones que pueden aumentar o reducir los riesgos. La seguridad debe

convertirse en una responsabilidad compartida durante todas las etapas del desarrollo.

Desarrollo

Cambio de paradigma

En la ingeniería de software, la seguridad suele revisarse al final, poco antes del despliegue.

Esta práctica resulta insuficiente ante amenazas cada vez más sofisticadas, cuya creación

también puede verse facilitada por la inteligencia artificial.

Tener un software seguro es una necesidad que se debe abordar desde su planeación, y no

como un agregado extra. Es necesario aplicar un enfoque proactivo, y no reactivo, en la seguridad para evitar las pérdidas de tiempo y dinero que pueden surgir por la necesidad de

aplicar un parche en un sistema que ya se encuentra en fases avanzadas de creación.

La responsabilidad compartida

Durante el desarrollo de un software, la seguridad no es algo que sea abordado únicamente

por un especialista. Por el contrario, es una cultura que debe aplicarse durante todo el ciclo de

vida del software por todos los involucrados en su creación. La seguridad comienza desde

que el analista toma las necesidades y las convierte en requerimientos técnicos. Se definen las

distintas reglas de negocio y las restricciones al acceso a la información desde el primer

momento; luego, con esta información, se toman decisiones sobre la estructura del software.

Con la implementación de los asistentes de inteligencia artificial y la generación automática

de código, el rol del programador cambia a dirigir y validar el código generado. Utilizar dicho

código sin revisarlo genera vulnerabilidades, cuya culpa recae sobre el programador. Por lo

tanto, el programador es responsable de leer, probar y validar la seguridad de cada parte del

código durante la creación.

El ciclo finaliza con el paso a producción. Aunque se cuente con despliegues automatizados,

se requiere de una supervisión constante para revisar permisos, configuraciones, credenciales

y secretos que puedan estar accesibles para todos.

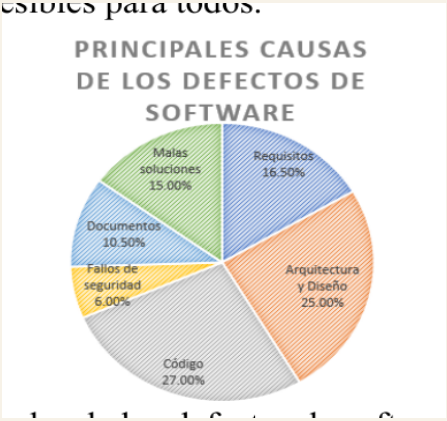


Figura 1. Causas principales de los defectos de software. Fuente: Jones (2018).

Monitoreo y continuidad

La seguridad tampoco termina cuando el sistema llega a producción. Es necesario monitorear

registros, configuraciones, accesos y comportamientos anómalos para detectar incidentes oportunamente. La información obtenida debe utilizarse para corregir defectos, actualizar los

controles y evitar que los mismos problemas vuelvan a presentarse. Además, deben existir

copias de seguridad, procedimientos de recuperación y responsables definidos.

Conclusión

La seguridad debe integrarse desde el inicio, pues muchas decisiones que protegen un sistema

se toman antes de escribir el código. Considerarla desde los requerimientos y la arquitectura

reduce vulnerabilidades y evita correcciones costosas. Esta responsabilidad corresponde a

todos los participantes del ciclo de vida del software. La inteligencia artificial facilita la

generación de código, pero sus resultados requieren revisión, pruebas y supervisión humana.

Después del despliegue, el monitoreo continuo ayuda a detectar comportamientos anómalos,

corregir defectos y responder ante incidentes. Las copias de seguridad y los procedimientos

de recuperación también mantienen la continuidad del servicio. Por ello, la seguridad desde el

diseño es necesaria para desarrollar sistemas confiables y protegerlos durante su operación.

Referencias

Jones, Capers. Applied Software Measurement: Global Analysis of Productivity and Quality.

CRC Press, 2018.

National Institute of Standards and Technology (NIST). The NIST Cybersecurity Framework

(CSF) 2.0. NIST Cybersecurity White Paper (CSWP) 29: U.S. Department of Commerce,

2024. https://doi.org/10.6028/NIST.CSWP.29.

National Institute of Standards and Technology (NIST). Secure Software Development

Practices for Generative AI and Dual-Use Foundation Models. NIST SP 800-218A. 2024.

https://doi.org/10.6028/NIST.SP.800-218A.

ARTÍCULO · ESTUDIANTE

¿Cómo evitar que la responsabilidad compartida se diluya en equipos de desarrollo que aplican seguridad desde el diseño?

Autores: Marcos Geovani Barrios López; Josue Daniel Chávez Portillo

Palabras clave: Seguridad desde el diseño, responsabilidad compartida, desarrollo de software, trazabilidad

Introducción

La seguridad de un sistema no comienza cuando el software está terminado, sino desde las primeras

decisiones sobre análisis y diseño. Estas decisiones influyen en los riesgos y controles del sistema

a lo largo del desarrollo. Por eso, los involucrados en el proceso de construcción del sistema deben

participar para que la seguridad sea una responsabilidad compartida desde el principio y así la

seguridad del sistema no recaiga en una sola persona.

La participación de todo el equipo no garantiza que cada decisión de seguridad tenga una persona

responsable en particular. Si las tareas no se asignan claramente, especialmente cuando se utiliza

IA como herramienta de apoyo, la responsabilidad puede diluirse hasta el punto en que nadie sea

responsable. Por lo tanto, debemos identificar las brechas en la responsabilidad y remediarlas antes

de continuar con el desarrollo.

Desarrollo

La distribución de la seguridad entre múltiples personas solo funcionará si todos entienden qué

hacer y hasta qué punto tienen autoridad para tomar decisiones. Debido a la mala comunicación

de la distribución, ciertos riesgos pueden pasar desapercibidos porque todos creen que alguien más

se hará cargo de ellos. Por lo tanto, NIST CSF 2.0 establece que las funciones, responsabilidades

y autoridades de seguridad deben ser definidas y comunicadas adecuadamente para poder responsabilizar a las personas.1

Esto no implica que todos los roles de seguridad se le asignen a una sola persona, sino que significa

la asignación de tareas específicas. El analista identificará las amenazas y los requerimientos; el

arquitecto diseñará el control; el programador lo implementará; y el revisor lo verificará. En el

1 Pascoe, Quinn y Scarfone, The NIST Cybersecurity Framework, 17. artículo actual se discute esta asignación desde las siguientes cuatro perspectivas: técnica, la cual es responsable de ejecutar y verificar el control; organizativa, la cual es responsable de asignar las

tareas; ética, la cual es responsable de analizar el impacto de las decisiones; y legal, la cual es responsable de verificar los requerimientos legales. NIST SSDF ayuda en la asignación de tareas.2

Según OWASP SAMM, el Security Champion es responsable de liderar a su equipo, pero no de realizar todas las tareas.3

Para evitar cualquier dilución de la responsabilidad, proponemos una responsabilidad rastreable,

la cual consta de tres componentes: la parte responsable, el revisor y la evidencia. La parte

responsable lleva a cabo la acción; el revisor asegura su cumplimiento; y la evidencia permite

probar el resultado. Si falta cualquiera de estos componentes, existe un vacío, el cual debería ser

llenado. El NIST SSDF respalda nuestro enfoque al sugerir que se cree evidencia y que se registren las decisiones durante el proceso de desarrollo.4

No es suficiente identificar la falta de responsabilidad cuando el proyecto puede continuar sin

llenar el vacío. En este sentido, proponemos puntos de control antes de la finalización de cada

actividad o fase. Si falta una parte responsable, un revisor o una prueba de que la tarea se ha

completado, esta deberá quedar pendiente. El NIST SSDF respalda este proceso de revisión y documentación al recomendar controles y el registro de las decisiones durante el desarrollo.5 La

Elemento	Preguntas de control	Condición para continuar
Parte responsable	¿Quién realiza la tarea o toma la decisión?	Existe una persona asignada.
Revisor	¿Quién revisa el resultado?	La revisión se llevó a cabo y fue registrada.
Evidencia	¿Existe alguna evidencia de esto?	Existen pruebas o registros.
Estado del control	¿Se han cumplido los tres elementos?	Se avanza únicamente cuando todo esté completado.

Tabla 1. Elementos de verificación en un punto de control. Fuente: elaboración propia.

Este principio también es válido si la inteligencia artificial es utilizada por el grupo como un

elemento de apoyo. Por ejemplo, si la inteligencia artificial escribe el código de un módulo de

autenticación, entonces el desarrollador tiene la responsabilidad técnica de verificar la seguridad

de este fragmento de código; el líder tiene la responsabilidad organizativa de coordinar su revisión;

el equipo analiza el impacto potencial en el usuario desde la perspectiva ética; mientras que el

oficial de cumplimiento revisa los temas legales pertinentes. La recomendación solo será aceptada

si hay una persona responsable, un revisor y evidencia de las pruebas realizadas.

Conclusión

Al delegarse las responsabilidades a ciertas personas, será importante mantener la responsabilidad

compartida. La participación de todos los miembros del grupo debería ir de la mano con

asignaciones claras para evitar acciones sin responsable. Las perspectivas técnica, organizativa,

ética y legal nos ayudan a identificar quiénes participan en cada decisión y cómo deben responder.

La identificación de vacíos en las responsabilidades nos ayuda a llenarlos antes de que surja algún

problema. Por lo tanto, cada decisión debe tener una persona responsable, un revisor y evidencia.

Los puntos de control permiten verificar estos requisitos. Este enfoque también se aplica cuando

la inteligencia artificial se utiliza como ayuda, pues sus sugerencias deben de ser analizadas de

manera critica y aprobadas.

Referencias bibliográficas

OWASP Foundation. “SAMM Agile Guidance.” OWASP Software Assurance Maturity Model.

Acceso el 20 de agosto de 2026 https://owaspsamm.org/docs/guides/agile/?.

Pascoe, Cherilyn, Stephen Quinn y Karen Scarfone. The NIST Cybersecurity Framework (CSF)

2.0. NIST Cybersecurity White Paper 29. Gaithersburg, MD: National Institute of Standards and

Technology, 2024. https://doi.org/10.6028/NIST.CSWP.29.

Scarfone, Karen, Murugiah Souppaya y Donna Dodson. Secure Software Development Framework

(SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST

Special Publication 800-218. Gaithersburg, MD: National Institute of Standards and Technology,

2022. https://doi.org/10.6028/NIST.SP.800-218.

ARTÍCULO · ESTUDIANTE

Compila hoy, rompe mañana: los problemas de compilar lo que la IA escribe

Autores: Kevin Josue Santos Salazar; Moisés Iván Daniel Alvarado García

Palabras clave: compilación, mantenibilidad, inestabilidad, despliegue continuo, logs.

Introducción

El código generado por inteligencia artificial ya no es novedad. Actualmente los agentes de

IA generan una gran parte de los commits diarios en proyectos reales y repositorios de todo

tipo, desde herramientas pequeñas hasta sistemas con miles de usuarios. Y con eso, el debate

de la calidad del código sigue detenido en el mismo punto, si el código compila y funciona al

entregarse.

Este enfoque ignora la etapa más costosa, que es el mantenimiento a largo plazo. Un

programa se escribe una vez, pero se mantiene durante años y la mantenibilidad no se

demuestra simplemente haciendo un par de tests que pasan en verde. A partir de un estudio

sobre repositorios reales y de un informe anual de la industria, este artículo analiza qué ocurre

con ese código meses después de haberse integrado.

Desarrollo

Hacer las cosas a la carrera no sale bien. En programación, arreglar algo rápido para

entregarlo hoy genera problemas que alguien tendrá que resolver mañana. La inteligencia

artificial no inventó este problema, pero sí tiene una capacidad impresionante para acumular

este tipo de dificultades y aumentar el desorden que se genera.

El reporte DORA de 2025 reveló que los equipos terminan tareas más rápido, pero también

incrementa la inestabilidad en la entrega de software. El código se genera tan rápido que los

pipelines de integración y despliegue de muchas empresas no han evolucionado lo suficiente

para procesar grandes volúmenes de cambios de manera segura. Como resultado, aumentan

los fallos en los despliegues y la necesidad de rehacer el trabajo.

Al revisar 623 millones de modificaciones de código entre 2023 y 2026 que fueron los años

en que la IA llegó a hacer más commits, el estudio de GitClear mostró que la calidad se cayó

en todos los frentes:

Señal estructural medida Variación

Llamadas a funciones entre archivos -35%

Líneas movidas por refactorización -70%

Cambios sobre código con más de doce meses de antigüedad -74%

Bloques de código duplicados +81%

Señal estructural medida	Variación
Llamadas a funciones entre archivos	-35%
Líneas movidas por refactorización	-70%
Cambios sobre código con más de doce meses de antigüedad	-74%
Bloques de código duplicados	+81%

Tabla 1. Variación de señales de mantenibilidad, 2023-2026. Datos de GitClear.

Los resultados muestran que el problema no está solamente en cuánto código puede

producirse, sino en lo que ocurre después de generarlo. La reducción de la refactorización y

el aumento del código duplicado sugieren que se dedica menos esfuerzo a reorganizar y

mejorar lo existente.

Los datos de GitClear sobre menor refactorización y mayor duplicación muestran que parte

del trabajo que se evita al inicio puede trasladarse al mantenimiento posterior. Cuando dichas

soluciones rápidas se acumulan, terminan convirtiéndose básicamente en deuda técnica que

posteriormente deberá resolverse.

Además, la IA crea la ilusión de corrección pero por dentro trae problemas escondidos, los

desarrolladores se enfrentan a problemas que van desde funcionalidades que no sirven o no

hacen nada y el borrado silencioso de pruebas unitarias, hasta la caída de servicios o todo el

sistema.

Debido a esto, el trabajo del desarrollador no desaparece, sino que cambia en vez de escribir

código, ahora dedica más tiempo a corregir errores y validar logs. Por esa falta de confianza

se entiende por qué los programadores dicen que no le tienen fe a la calidad del código hecho

por IA.

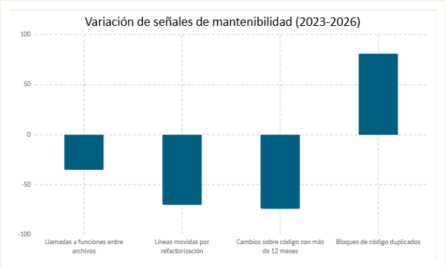


Figura 1. Variación de señales de mantenibilidad (2023-2026). Fuente: elaboración propia con base en datos de GitClear (2026).

La figura 1 lo deja claro, ninguna barra queda cerca de cero, lo que señala un cambio de

hábito sostenido y no una variación menor. Al mismo tiempo, creció 47 % la costumbre de

esconder errores con parches rápidos, es decir, silenciando los fallos en lugar de arreglarlos

de raíz.

Conclusiones

La inteligencia artificial ha beneficiado en muchos aspectos, su uso para generar código

puede ayudar en gran medida en el desarrollo de las funcionalidades con mayor rapidez, pero

que un programa funcione, en su entrega, no necesariamente significa que la calidad sea la

óptima según los requerimientos y menos a largo plazo, la IA se ha caracterizado por

solucionar rápido. Esto de hecho puede incurrir en que el ahorro del tiempo inicial, luego se

deba invertir en corregir errores. Por esta razón debe ser una herramienta de apoyo pero no un

reemplazo de la revisión de un desarrollador. El beneficio real de la IA dependerá no solo de

cuánto código se pueda producir, sino la mantenibilidad que puede llegar a tener en el tiempo.

Esto permite mantener abierta la discusión sobre si el ahorro de tiempo que ofrece la

inteligencia artificial justifica las dificultades que pueden aparecer después.

Referencias bibliográficas

GitClear. “The Maintainability Gap: 2026 AI Code Quality Research”. GitClear, 2026. https://www.gitclear.com/the_ai_code_quality_maintainability_gap.

Google Cloud y DORA. State of AI-assisted Software Development 2025. Google, 2025. https://services.google.com/fh/files/misc/2025_state_of_ai_assisted_software_development.p df.

Perry, Neil, et al. “Do Users Write More Insecure Code with AI Assistants?” ACM, 2023. https://doi.org/10.1145/3576915.3623157.